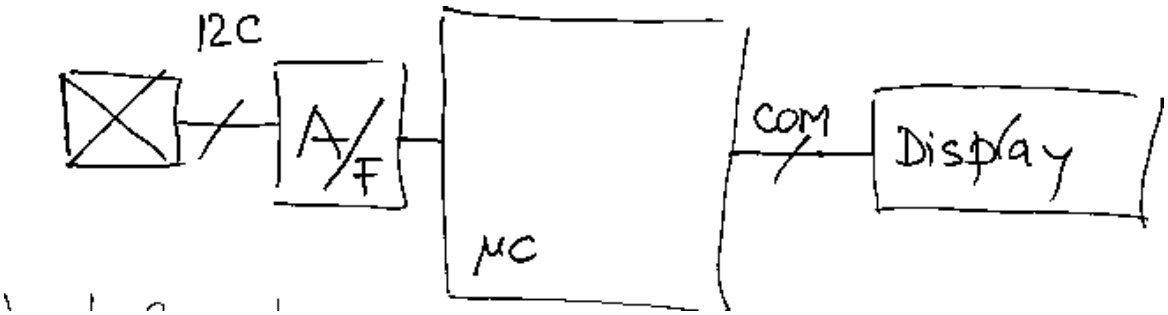
Proiect Senzori și Traductoare Sistem integrat de achiziție și afișare a temperaturii utilizând senzor PT100 și microcontroler PIC16F877A

Proiect realizat de

Ciobotaru Ionuţ-Andrei

Grupa 5405



Cuprins:

1. Stadiul Actual
2. Arhitectura
3. Schema electronică
4. Editarea codului
5. Testare/Validare
6. Concluzii
7. Bibliografie
8. Anexă. Codul sursă al proiectului
9. Anexă. Imagine 3D a circuitului

Capitolul I. Stadiul Actual

PT100 este un senzor de temperatură de tip RTD (Resistance Temperature Detector), bazat pe variația rezistenței electrice a platinei în funcție de temperatură.

Referitor la stadiul actual al tehnologiei, există numeroase brevete care abordează metode moderne de măsurare a temperaturii folosind senzori RTD de tip PT100. Aceste brevete tratează aspecte precum îmbunătățirea acurateței, compensarea automată a erorilor, circuite avansate de măsurare sau integrarea în sisteme industriale automate.

Tipul senzorului PT100 oferă două informații importante: prefixul „Pt” indică faptul că elementul sensibil este realizat din platină, iar sufixul „100” reprezintă valoarea rezistenței electrice a senzorului la 0°C, respectiv 100 Ω. Există și alte variante de senzori RTD care folosesc materiale diferite, precum nichelul (Ni) sau cuprul (Cu), precum și valori nominale distincte ale rezistenței, cum ar fi 50 Ω, 500 Ω sau 1000 Ω, ceea ce permite identificarea acestora prin denumiri precum Cu100, Ni120 sau Pt1000 [1].

Senzorul de temperatură PT100 este un detector de temperatură prin rezistență (RTD) care funcționează pe baza variației rezistenței electrice a platinei în funcție de temperatură. Pe măsură ce temperatura mediului crește, rezistența elementului din platină crește într-o manieră aproape liniară, având valoarea de 100 Ω la 0°C și un coeficient de variație de aproximativ 0,00385 Ω/Ω/°C. Datorită acestei relații previzibile între temperatură și rezistență, temperatura poate fi determinată prin măsurarea rezistenței senzorului și aplicarea relației matematice specifice sau a tabelelor standardizate PT100. Proprietățile fizice ale platinei, precum stabilitatea, linearitatea și inerția chimică, fac ca senzorul PT100 să fie precis, repetabil și potrivit pentru aplicații de măsurare și control industrial [2].

Senzorii de temperatură PT100 oferă multiple avantaje care îi fac foarte potriviți pentru aplicații industriale: au precizie și acurațe ridicată, datorită modului predictibil în care rezistența din platina se schimbă cu temperatura; funcționează eficient pe un interval larg de temperaturi (de la aproximativ −200 °C până la +850 °C); sunt construiți pentru a rezista în condiții industriale dure (moist, praf, substanțe corozive) și pot fi încapsulați pentru durabilitate sporită; se calibrează ușor și sunt interșanjabili conform standardelor internaționale; pot fi integrati ușor în diverse sisteme industriale datorită formelor și ieșirilor standardizate; și oferă stabilitate și durabilitate pe termen lung, cu minime deviații în timp, ceea ce îi face alegerea ideală pentru monitorizarea continuă a temperaturii în procese critice [3] .

Senzorii Pt100 pot fi conectați prin mai multe tipuri de circuite, în funcție de necesarul de precizie și de modul de compensare a rezistenței firelor. Circuitele cu 2 fire sunt cele mai simple și frecvent utilizate, însă rezistența firelor poate introduce erori în măsurători dacă nu este compensată prin calibrare; aceste circuite sunt adesea folosite în senzorii wireless, unde semnalul este digitalizat aproape de senzor pentru a evita degradarea semnalului [4].

Circuitele cu 3 fire adaugă un fir suplimentar care permite măsurarea și compensarea rezistenței firelor, îmbunătățind acurațeța măsurătorilor [4].

Pentru cea mai mare precizie se folosesc circuitele cu 4 fire, în care patru fire măsoară și compensează rezistența conductorilor, asigurând rezultate precise indiferent de lungimea sau calitatea cablurilor [4].

De-a lungul timpului, au fost dezvoltate și brevetate numeroase proiecte și soluții care utilizează senzorii PT100. Acestea includ metode inovatoare pentru măsurarea temperaturii, condiționarea semnalului și integrarea senzorilor în sisteme industriale, reflectând stadiul actual al tehnologiilor și diversitatea abordărilor în domeniu.

Printre soluțiile brevetate se regăsesc:

- Soluție care asigură măsurarea precisă a unui senzor PT100 în aplicații cu cabluri foarte lungi, soluție care elimină erorile generate de rezistența firelor, soluție care depășește limitările metodelor în 2 și 3 fire și soluție care folosește configurația în 4 fire pentru a elimina complet influența cablului asupra măsurării temperaturii [5].
- Soluția care corectează eroarea de măsurare a senzorilor PT100 înainte de interpretarea rezultatelor, astfel încât diferențele de temperatură măsurate să reflecte realitatea fizică, nu erorile proprii ale senzorilor [6].
- Soluție care propune un sistem de măsurare și control al temperaturii cu PT100, folosind o punte electrică și un amplificator diferențial LM741 pentru eliminarea erorilor de cablu. ADC0808 corectează neliniaritățile, iar un microcontroler STC procesează datele și afișează temperatura în timp real. Sistemul permite setarea limitelor și comanda prin releu [7].
- Soluție care propune un sistem de măsurare a temperaturii cu PT100 bazat pe ADC-ul AD7711, care integrează atât condiționarea semnalului, cât și conversia analog–digitală, oferind totodată un curent constant pentru senzor. Utilizarea acestui ADC reduce complexitatea circuitului și îmbunătățește precizia. MCU-ul STC10F12 procesează datele și comunică prin interfață USB. Sistemul rezultat este compact, eficient și cu bună imunitate la interferențe [8].
- Investigarea erorile suplimentare apărute la măsurarea temperaturii cu senzori Pt100 montați pe elemente aflate în deflecție. În timpul testelor, s-au observat abateri ale temperaturii atunci când senzorul era simultan încălzit și deformat mecanic. Au fost realizate experimente de laborator și verificări termografice, demonstrând că deformarea influențează rezistența Pt100, generând erori ce trebuie compensate în aplicațiile practice [9].
- Soluție care utilizează un sistem de măsurare și control al temperaturii pe bază de PT100, integrând o punte de măsură, un amplificator diferențial LM741 pentru eliminarea erorilor de linie, un convertor ADC0808 pentru corectarea neliniarităților și un microcontroler STC pentru prelucrare și afișare în timp real. Arhitectura propusă crește precizia, stabilitatea și robustețea sistemelor industriale de control termic [10].
- Soluție care propune o metodă simplificată de testare a calității senzorilor PT100 prin determinarea coeficientului de temperatură al rezistenței, utilizând un sistem de achiziție bazat pe microcontroler, ADC de 24 biți, amplificare și divizor de tensiune. Abordarea permite evaluarea purității platinului și a erorilor de fabricație, oferind o alternativă accesibilă la metodele standard de calibrare [11]
- Soluție care descrie dezvoltarea unei unități capabile să simuleze un senzor PT100 pentru testarea sistemelor vehiculului, controlată prin CAN și capabilă să reproducă situații precum scurtcircuitul sau întreruperea. Simularea temperaturii este realizată prin potențiometre digitale conectate în paralel, iar

stările de defect sunt reproduse cu relee, permițând evaluarea răspunsului electronicii reale în condiții variate [12].

- Soluție care valorifică un sistem de măsurare de înaltă precizie pentru senzorul Pt100, cu etalonare, condiționare de semnal și achiziție multi-canal, utilizând amplificatoare operaționale MCP609 și platforma LabJack UE9. Metodologia permite compensarea erorilor, măsurarea simultană pe multiple canale și integrarea software pentru analiză avansată, contribuind la creșterea acurateței în aplicații educaționale și industriale [13].
- Soluție care presupune proiectarea unui modul de condiționare pentru senzorul PT100, utilizând o conexiune sub-trei-fire și un curent constant de precizie pentru reducerea erorilor. Configurația integrează amplificare și multiplexare, oferind măsurări multi-canal cu o acuratețe de ~0,1 °C, adecvată pentru aplicații industriale ce necesită stabilitate ridicată [14].
- Soluție care prezintă un sistem automatizat de blowdown pentru cazane industriale, bazat pe un modul de condiționare a semnalului și un microcontroler PIC16, care elimină necesitatea ciclului de purge și reduce erorile de operare. Soluția integrează măsurarea simultană a temperaturii și conductivității, optimizând timpii de reacție și menținând calitatea aburului. Abordarea crește eficiența procesului prin control precis al supapei și monitorizare în timp real [15].

Analizarea acestor brevete arată că senzorul PT100 este o soluție matură și folosită în numeroase aplicații moderne, însă majoritatea sistemelor comerciale sunt costisitoare sau complexe. Proiectul de față propune o implementare educațională și accesibilă folosind PIC16F877A și afișaj I2C, combinând principiile din literatura brevetată cu o soluție simplă și reproductibilă.

Capitolul 2. Arhitectura

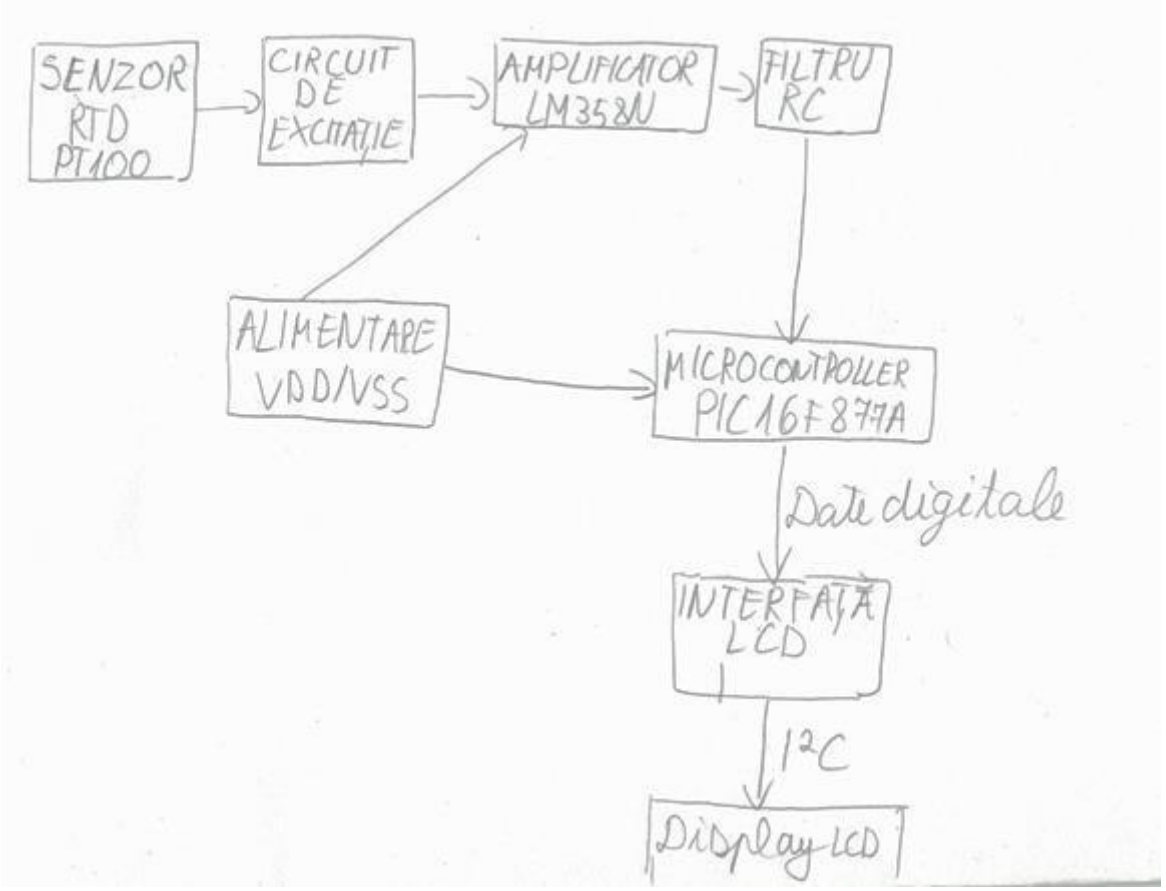


Figura 2.1. Schema Bloc a circutului

Circuitul propus pentru achiziția temperaturii este alcătuit dintr-un senzor RTD PT100, un amplificator operațional LM358N configurat într-o punte Wheatstone pentru convertirea variațiilor de rezistență în semnal de tensiune, un convertor analog-digital PCF8591 și un microcontroler PIC16F877A. Datele măsurate sunt transmise către un virtual terminal prin protocol UART, permițând vizualizarea și monitorizarea temperaturii în timp real.

Legătura dintre circuit și microcontrolerul PIC16F877A se realizează prin interfața I²C, utilizând convertorul analog-digital extern PCF8591 pentru achiziția semnalului de la senzorul PT100. În plus, pentru anumite măsurători se utilizează și convertorul analog-digital intern al PIC-ului, asigurând flexibilitate și posibilitatea de comparare a semnalelor pentru creșterea preciziei măsurătorilor.

Componentele circuitului. Amplificatorul operațional LM358N

LM358N este un amplificator operațional dual cu consum redus, care conține doi op-amp independenți într-un singur pachet. Acesta poate amplifica diferența de tensiune dintre intrările invertante și neinvertante, transformând astfel variațiile de semnal de la senzorul PT100 într-un semnal de tensiune corespunzător, care poate fi prelucrat de ADC. LM358N funcționează atât cu surse simple, cât și cu surse duale de alimentare, oferind o

câștig mare de tensiune și un interval larg de operare (3–32 V). [16] Datorită consumului redus de curent și stabilității bune la temperaturi uzuale de funcționare, este potrivit pentru aplicații industriale și pentru sisteme de achiziție a temperaturii, unde precizia și fiabilitatea semnalului condiționat sunt esențiale pentru vizualizarea valorilor pe un virtual terminal prin UART.

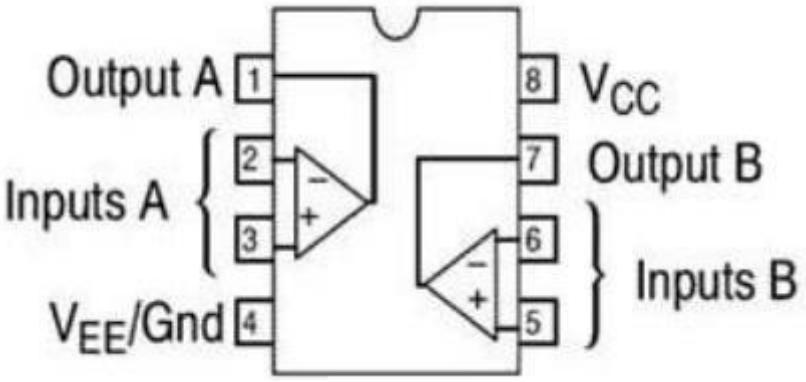


Figura 2.2. Pinout-ul amplificatorul operațional LM358N [16]

Puntea Wheatstone este un circuit utilizat pentru a transforma variațiile de rezistență ale senzorului PT100 în tensiune măsurabilă. PT100 fiind un senzor de temperatură cu rezistență variabilă, orice modificare a temperaturii se traduce printr-o schimbare a rezistenței sale. Într-o punte Wheatstone, această variație produce un dezechilibru de tensiune între două ramuri ale punții, generând astfel un semnal proporțional cu temperatura. Acest semnal poate fi apoi amplificat și transmis către microcontrolerul PIC16F877A sau către un ADC extern precum PCF8591, asigurând o măsurare precisă și stabilă a temperaturii.

PCF8591 este un convertor analog-digital și digital-analog pe un singur cip, cu 8 biți, care oferă patru intrări analogice și o ieșire analogică. Acesta comunică cu microcontrolerul prin interfața I^2C, permițând integrarea ușoară în sisteme de achiziție de date. Intrările analogice pot fi configurate ca single-ended sau diferențiale, iar pe cip există un circuit de track-and-hold pentru a menține valoarea tensiunii în timpul conversiei. Convertorul A/D folosește tehnica successive approximation pentru a transforma semnalul analogic în cod binar de 8 biți, iar conversia D/A este realizată printr-un lanț de rezistențe și un amplificator buffer. PCF8591 permite auto-incrementarea canalelor pentru achiziții multiple și poate folosi fie o tensiune de referință externă, fie alimentarea ca referință internă, oferind flexibilitate și precizie în achiziția semnalelor de la senzorii PT100 [17].

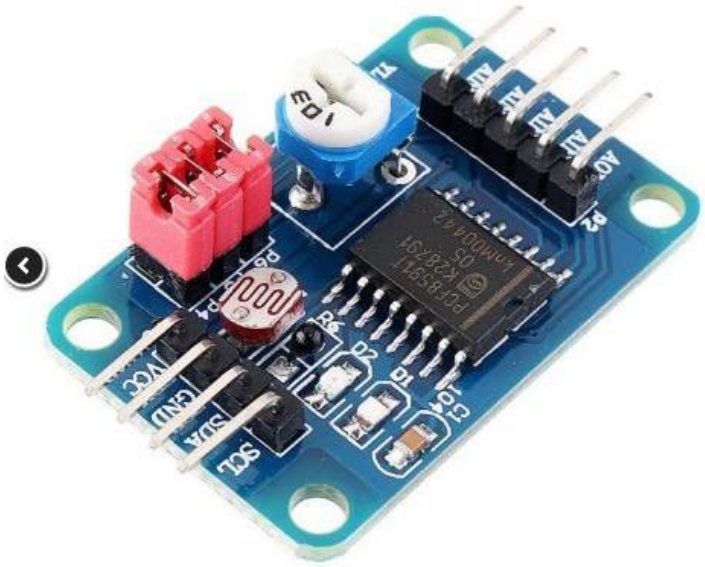


Figura 2.3 PCF8591 [18]

Convertorul PCF8591 este conectat la microcontroler prin interfața I²C, folosind pinii RC3 (SCL) și RC4 (SDA) ai PIC16F877A. Pentru a asigura funcționarea corectă a magistralei I²C, cele două linii sunt prevăzute cu rezistențe de pull-up conectate la sursa de alimentare, menținând nivelurile logice stabile și prevenind stările flotante.

PIC16F877A este un microcontroler cu arhitectură RISC de înaltă performanță, cu memorie Flash programabilă (8K x 14 biți), 368 octeți RAM și 256 octeți EEPROM pentru stocarea datelor. Microcontrolerul operează la o frecvență maximă de 20 MHz, cu un ciclu de instrucțiune de 200 ns, și suportă un set redus de instrucțiuni (doar 35), ceea ce simplifică programarea și optimizează viteza de execuție [19].

Microcontrolerul PIC16F87XA dispune de o unitate centrală (CPU) RISC, echipată cu o unitate aritmetică și logică (ALU) de 8 biți, registrul de lucru W și registrul de stare STATUS [19]. Memoria acestuia include Flash pentru stocarea programului, RAM pentru date temporare și EEPROM pentru date persistente [19]. Dispozitivul oferă cinci porturi de I/O (A, B, C, D, E) pentru conectarea senzorilor și actuatoarelor [19].Pentru gestionarea timpului și generarea semnalelor PWM sunt disponibili Timer0 (8 biți), Timer1 (16 biți) și Timer2 (8 biți) [19]. Microcontrolerul include și module speciale, precum Capture/Compare/PWM (CCP1 și CCP2) pentru controlul precis al semnalelor, USART pentru comunicație serială asincronă, MSSP pentru comunicație SPI sau I²C (utilizat, de exemplu, cu PCF8591), precum și un comparator analogic și un convertor A/D de 10 biți cu 8 canale, care permite citirea senzorilor analogici, cum ar fi PT100 prin intermediul amplificatorului LM358N [19].

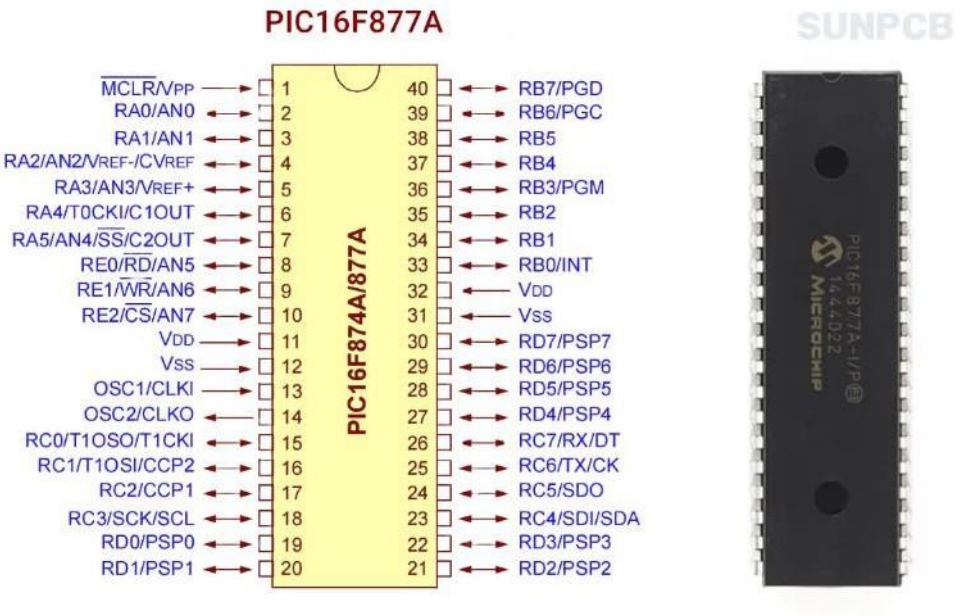


Figura 2.4. Pinout-ul lui PIC16F877A [20]

Protocoalele utilizate în circuit: I²C și UART

I²C (Inter-Integrated Circuit) este un protocol de comunicație serială folosit pentru a conecta mai multe dispozitive electronice, cum ar fi senzori, convertoare ADC/DAC sau afișaje, la un microcontroler sau între mai multe microcontrolere. Protocolul utilizează doar două fire: SDA (Serial Data) pentru transmiterea și recepția datelor și SCL (Serial Clock) pentru semnalul de ceas. Comunicarea se realizează bit cu bit și este sincronă, datele fiind transferate sincron cu semnalul de ceas, controlat de master [21].

Transferul de date I²C se realizează prin mesaje, care sunt împărțite în cadre. Fiecare mesaj include un start condition, care marchează începutul comunicării prin trecerea liniei SDA de la HIGH la LOW înainte ca SCL să fie LOW. Urmează adresa slave-ului, pe 7 sau 10 biți, și un bit Read/Write care indică direcția transferului. Cadrele de date sunt transmise pe 8 biți, MSB mai întâi, fiecare fiind urmat de un bit ACK/NACK pentru confirmarea recepției. La sfârșitul comunicării, masterul trimite un stop condition, prin trecerea liniei SDA de la LOW la HIGH după ce SCL este HIGH [22].

Protocolul suportă configurații cu mai mulți masteri și slave-uri. Dacă doi masteri încearcă să transmită simultan, protocolul folosește arbitrajul pe linia SDA pentru a evita conflictele. Datorită conexiunilor open-drain și efectului wired-AND, linia rămâne LOW dacă oricare dispozitiv o trage la LOW. Masterii monitorizează magistrala și cedează controlul dacă detectează că SDA este LOW atunci când încearcă să transmită HIGH, garantând astfel că doar un master câștigă magistrala fără a întrerupe comunicarea [21].

SCL este controlat în mod normal de master, dar slave-urile pot folosi mecanismul de clock stretching pentru a încetini transferul și a câștiga timp pentru procesarea datelor. În acest caz, slave-ul menține SCL LOW, iar masterul nu poate continua până când ceasul este

eliberat. Astfel, protocolul devine flexibil și sigur pentru periferice cu viteze diferite de procesare [21].

Liniile SDA și SCL sunt bidirecționale și conectate la rezistențe de pull-up către Vcc. Aceste rezistențe asigură că liniile ating nivelul logic HIGH atunci când niciun dispozitiv nu le trage LOW. Dimensionarea corectă a rezistențelor este esențială pentru a asigura viteza dorită de transfer și integritatea semnalului, mai ales pe magistrale cu capacitânță parazitară mare [21].

I²C oferă avantaje precum folosirea a doar două fire, suport pentru multiple mastere și slave-uri, confirmarea transferului prin ACK/NACK și hardware mai simplu pentru multiple dispozitive. Dezavantajele includ viteze mai mici decât SPI, limite de 8 biți pe cadre și o implementare hardware mai complexă pentru sisteme cu mai mulți masteri [22].

UART (Universal Asynchronous Receiver/Transmitter) este un protocol de comunicație serială între două dispozitive, care funcționează asincron, fără ceas comun. Folosește două fire: Tx pentru transmitere și Rx pentru recepție, iar sincronizarea se face prin setarea aceluiași baud rate pe ambele dispozitive. [23]

Datele sunt trimise în pachete formate din start bit, cadru de date (5-8 biți), bit de paritate opțional și stop bits. Start bit-ul semnalează începutul transmisiei, iar stop bit-ul marchează sfârșitul. [23]

Bit-ul de paritate ajută la detectarea erorilor cauzate de interferențe sau nealinierea baud rate-urilor, asigurând transmiterea corectă a datelor. [23]

Ambele dispozitive trebuie să aibă același baud rate, număr de biți de date și stop bits. Configurarea corectă se face folosind registre specifice, precum UART_DIV și UART_FBR, în funcție de ceasul periferic. [23]

UART este simplu, folosește doar două fire și este folosit pentru debug, actualizări software, loguri de producție și teste. Implementarea unui protocol de cadru poate adăuga securitate prin headere, trailere și verificare CRC. [23]

Capitolul 3. Schema circuitului

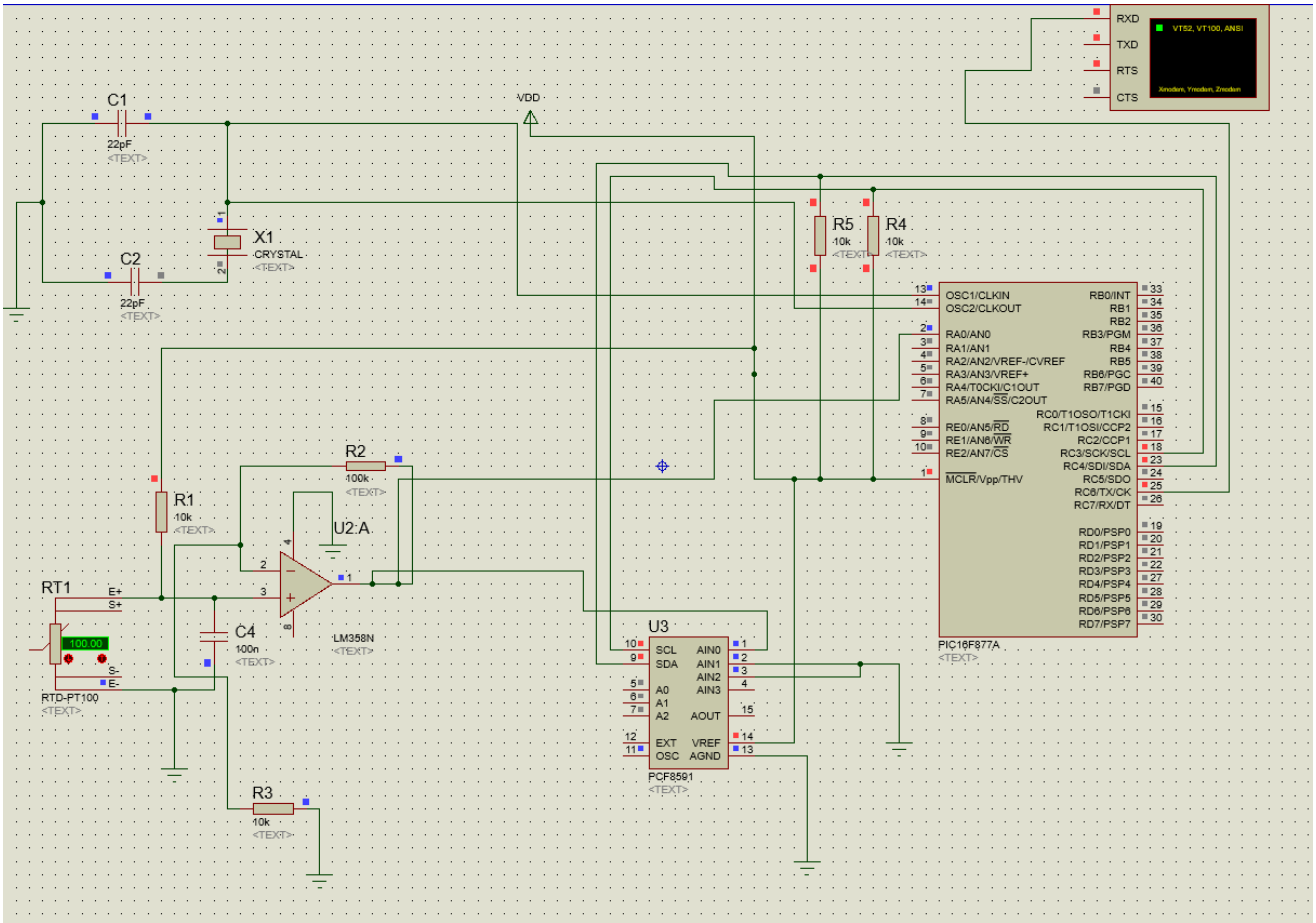


Figura 3.1. Schema sistemului integrat cu RTD PT-100

Funcţionarea circuitului:

Senzorul RTD PT100 este integrat într-o configurație de punte Wheatstone împreună cu rezistențele R1, R2 și RT1. La variația temperaturii, rezistența PT100 se modifică, creând un dezechilibru al punții care generează o tensiune diferențială de ordinul milivolților între cele două noduri centrale ale punții.

Senzorul RTD PT100 este integrat într-o punte Wheatstone, împreună cu rezistențe fixe de precizie. În această configurație, variația rezistenței PT100 determină un dezechilibru al punții, generând o tensiune diferențială între cele două noduri centrale. Această tensiune este direct dependentă de temperatură, dar are o amplitudine redusă, de ordinul milivolților.

Ieșirile punții Wheatstone sunt conectate la intrările amplificatorului operațional LM358N, configurat pentru amplificarea tensiunii diferențiale. Amplificatorul crește nivelul semnalului și îl adaptează domeniului de tensiune necesar pentru conversia analog-digitală.

Alimentarea cu sursă unică permite funcționarea corectă a amplificatorului în raport cu nivelurile de masă și tensiune utilizate în circuit.

Semnalul analogic amplificat este distribuit către două intrări de conversie. Prima conexiune este realizată către una dintre intrările analogice ale convertorului PCF8591, permițând achiziția semnalului printr-un ADC extern. A doua conexiune este realizată către o intrare analogică a microcontrolerului PIC16F877A, utilizând convertorul analog-digital intern. Această ramificare a semnalului este vizibilă în schemă și permite utilizarea simultană a ambelor convertoare.

Convertorul PCF8591 este conectat la microcontroler prin magistrala I²C, folosind liniile SDA și SCL, prevăzute cu rezistențe de pull-up la tensiunea de alimentare. Aceste rezistențe asigură stabilitatea nivelelor logice și funcționarea corectă a comunicației pe magistrală.

Microcontrolerul PIC16F877A primește astfel semnalul de temperatură atât direct, prin intrarea analogică, cât și indirect, prin convertorul extern. În schemă este evidențiat și modulul UART, utilizat pentru transmiterea rezultatelor către terminalul virtual, prin pinii dedicați de transmisie și recepție.

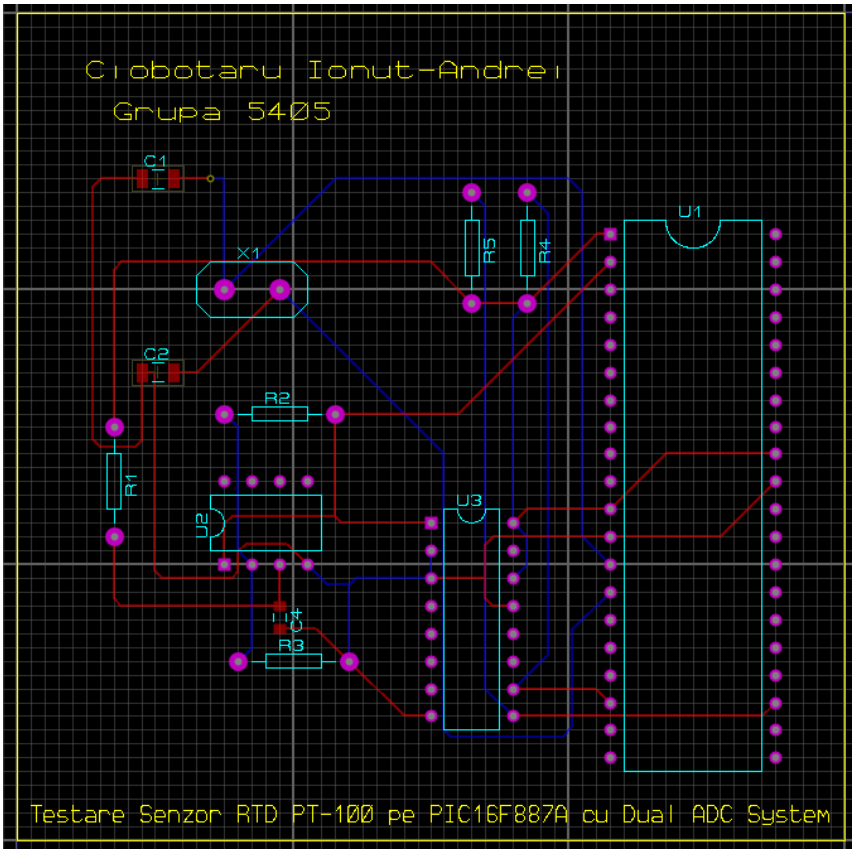


Figura 3.2. Schema PCB a sistemului, excluzând PT-100, al cărui footprint nu există în Proteus

Capitolul 4. Editarea codului

Sistemul dual-ADC folosește două metode de măsurare pentru a obține rezultate mai bune decât ar oferi o singură sursă. ADC-ul intern de 10 biți citește mai precis valorile mici, având patru ori mai multe nivele decât PCF8591 cu 8 biți, în timp ce convertorul extern poate măsura pe mai multe canale și este izolat prin magistrala I²C. Funcția smart compară cele două citiri și alege rezultatul cel mai de încredere, folosind ADC-ul intern când valorile sunt între punctele de calibrare și PT100 când se află exact pe punctele cunoscute. Media ponderată combină cele două măsurători în funcție de cât de sigure sunt, obținând o temperatură finală mai stabilă și mai apropiată de valoarea reală. Acest sistem se adaptează singur la fiecare citire, oferind precizie mare pe tot intervalul de temperatură și eliminând erorile care ar apărea dacă s-ar folosi doar o singură metodă de măsurare.

Codul sursă este organizat modular în secțiuni funcționale distincte: configurare hardware și constante de calibrare, funcții UART pentru transmisia datelor către terminal, funcții I²C pentru comunicația cu PCF8591, funcții ADC intern pentru convertorul de 10 biți al microcontrolerului, algoritmi de conversie care transformă valorile ADC în temperatură prin interpolare liniară, modul de fuziune inteligentă care combină cele două măsurători calculând niveluri de încredere, funcții de afișare pentru formatarea datelor în tabel, și bucla principală care coordonează execuția ciclică a achiziției și afișării temperaturii.

Directivele pragma config stabilesc parametrii de funcționare ai microcontrolerului: oscilatorul HS pentru cristalul de 8MHz, dezactivarea watchdog timer-ului (WDTE=OFF), dezactivarea funcțiilor de protecție a memoriei (CP, CPD, WRT) și a programării la tensiune joasă (LVP), asigurând o configurare simplă și stabilă pentru aplicație.

Funcția UART_Init configurează pinii RC6 ca ieșire pentru transmisie și RC7 ca intrare pentru recepție, setează registrul SPBRG la valoarea 25 pentru obținerea vitezei de 9600 baud la frecvența de 8MHz, și activează modulul de transmisie prin registrele TXSTA și RCSTA. Funcția UART_Write transmite un singur caracter așteptând setarea flag-ului TXIF înainte de a încărca caracterul în registrul TXREG. Funcția UART_Write_String trimite șiruri de caractere apelând repetat UART_Write pentru fiecare caracter până la întâlnirea terminator-ului de șir. Funcția UART_Write_Int convertește valorile întregi în format text folosind sprintf și le transmite prin UART_Write_String, permițând afișarea numerică a rezultatelor în terminal.

Funcția I²C_Init configurează pinii RC3 și RC4 ca intrări pentru liniile SCL și SDA, setează registrele SSPSTAT, SSPCON și SSPADD pentru modul master cu frecvența de 100kHz. Funcția I²C_Start generează condiția de start prin setarea bitului SEN și așteaptă finalizarea acesteia. Funcția I²C_Stop generează condiția de stop prin setarea bitului PEN. Funcția I²C_Write transmite un byte încărcându-l în SSPBUF, așteaptă setarea flag-ului BF și verifică finalizarea transmisiei prin I²C_Wait. Funcția I²C_Read activează recepția prin RCEN, citește byte-ul din SSPBUF când BF este setat, și trimite acknowledge (ACKDT=0) sau not-acknowledge (ACKDT=1) în funcție de parametrul primit, controlând astfel fluxul de date pe magistrală.

Funcția PCF8591_Read_Channel construiește byte-ul de control prin combinarea valorii 0x40 cu numărul canalului selectat, generează condiția de start, trimite adresa de

scriere a PCF8591 urmată de byte-ul de control, apoi oprește și reinițiază comunicația pentru citire. După trimiterea adresei de citire, funcția primește două valori consecutive, prima fiind ignorată deoarece PCF8591 returnează mai întâi rezultatul conversiei anterioare, iar a doua valoare reprezintă conversia curentă care este returnată. Funcția PCF8591_Read_Avg realizează medierea valorilor prin acumularea a 32 de citiri consecutive într-o variabilă long, cu pauze de 3ms între citiri, și returnează media prin împărțire cu 32 (deplasare la dreapta cu 5 biți), reducând astfel zgomotul și îmbunătățind stabilitatea măsurătorilor.

Funcția ADC_Internal_Init configurează pinul RA0 ca intrare prin setarea TRISA0, inițializează registrul ADCON0 cu valoarea 0x81 pentru activarea ADC-ului și selectarea canalului AN0, și setează ADCON1 la 0x8E pentru justificare la dreapta a rezultatului pe 10 biți și configurarea celorlalți pini ca digitali. Funcția ADC_Internal_Read pornește conversia prin setarea bitului GO din ADCON0, așteaptă finalizarea conversiei verificând același bit, și returnează rezultatul pe 10 biți format din concatenarea registrelor ADRESH și ADRESL. Funcția ADC_Internal_Read_Avg acumulează 32 de citiri consecutive cu pauze de 3ms între ele într-o variabilă long, apoi calculează media prin deplasare la dreapta cu 5 biți, oferind astfel valori stabile și reduse în zgomot pentru măsurarea precisă a temperaturii.

Funcțiile Get_Temp_Internal_Int și Get_Temp_PT100_Int implementează interpolarea liniară pe segmente între punctele de calibrare definite la 0°C, 25°C, 50°C, 75°C și 100°C, fiecare segment având propriile valori ADC corespunzătoare. Algoritmul identifică segmentul în care se află valoarea ADC citită prin comparații succesive, apoi calculează temperatura prin interpolare liniară între capetele segmentului, utilizând formula care determină raportul dintre distanța de la punctul inferior și lungimea segmentului. Temperatura este calculată înmulțită cu 100 pentru a păstra precizia zecimalelor fără a utiliza operații în virgulă mobilă, obținând astfel valori întregi care reprezintă grade Celsius cu două zecimale. Valorile calculate sunt limitate în domeniul 0-10000 (reprezentând 0.00°C până la 100.00°C) pentru a preveni rezultate eronate în afara intervalului de măsurare valid.

Funcția Get_Smart_Temperature evaluează încrederea în ADC intern verificând dacă valoarea citită se află într-o fereastră de ±2 unități în jurul punctelor de calibrare, acordând încredere maximă de 1.0 în acest caz sau 0.8 în afara acestor intervale, apoi amplifică această încredere cu factorul 4.0 datorită rezoluției superioare de 10 biți față de 8 biți. Pentru PT100, funcția acordă încredere maximă de 1.0 doar când valoarea ADC coincide exact cu un punct de calibrare, altfel încrederea scade la 0.2 din cauza rezoluției reduse și a incertitudinii în interpolări. Algoritmul calculează suma totală a încrederii de la ambele surse și determină identificatorul sursei: 3 pentru ambele pe calibrare (încredere maximă), 1 pentru dominanță ADC intern când încrederea sa este de peste două ori mai mare, sau 2 pentru medie ponderată în celelalte cazuri. Temperatura finală este calculată ca medie ponderată a celor două valori, fiecare înmulțită cu nivelul său de încredere și împărțită la suma totală de încredere, obținând astfel un rezultat optim care valorifică punctele forte ale fiecărei surse.

Funcția Print_Startup_Banner afișează la pornire un banner decorativ care prezintă titlul proiectului, tipul sistemului dual-ADC utilizat, și datele autorului cu grupa de studiu, oferind o interfață profesională în terminal. Funcția Print_Init_Status afișează progresul inițializării fiecărui modul prin afișarea numelui modulului urmat de puncte de completare și statusul "OK", creând o secvență vizuală clară a procesului de pornire a sistemului. Funcția Print_Table_Header creează antetul tabelului cu coloane delimitate prin linii și spații,

indicând ADC Intern, Temp (INT), ADC I2C, Temp PT100 și Temp Finala, structurând afișarea datelor într-un format ușor de citit. Funcția Print_Reading formatează fiecare citire pe o singură linie, afișând valorile ADC și temperaturile cu formatare adecvată a spațiilor pentru aliniere, temperaturile fiind afișate cu o zecimală, și adaugă indicatori de calitate la sfârșit: asteriscul (*) pentru precizie maximă când ambele ADC-uri sunt pe puncte de calibrare, tilda (~) pentru medie ponderată, sau spații pentru dominanță ADC intern.

```
============================================================
    TESTARE SENZOR RTD PT100 PE PIC16F877A
    Dual-ADC System: Internal 10-bit + PCF8591 I2C
------------------------------------------------------------
    Proiect realizat de: Ciobotaru Ionut-Andrei
                         Grupa 5405
============================================================

  [Comunicatie UART] ...... OK
  [Magistrala I2C] ........ OK
  [ADC External PCF8591] .. OK
  [ADC Intern PIC] ........ OK
  [Algoritm Fuziune] ...... OK

  >> Sistem operational <<
  >> PT100 conectat pe PCF8591 Canal 0 <<

-----------------------------------------------------------------
 ADC Intern | Temp (INT) | ADC I2C | Temp PT100 | Temp Finala
-----------------------------------------------------------------
    150     |   79.5C   |   38    |   87.5C    |   80.1C
    156     |   93.1C   |   39    |   100.0C    |   94.8C
    158     |   97.7C   |   39    |   100.0C    |   98.1C *
    159     |   100.0C   |   40    |   100.0C    |   100.2C
|
```

Figura 4.1. Interfaţa programului pentru măsurare a temperaturii

Capitolul 5. Testare.Validare

Validarea sistemului de măsurare a fost realizată prin testare experimentală pe întregul interval de temperatură, de la 0°C la 100°C, cu pas de 5°C între fiecare punct de măsurare. Pentru fiecare temperatură setată, au fost înregistrate valorile citite de ambele ADC-uri și temperatura finală calculată de algoritmul de fuziune inteligentă.

Datele au fost colectate și organizate într-o foaie de calcul Excel, permițând compararea sistematică între valorile așteptate și cele măsurate efectiv de sistem.

Analiza rezultatelor experimentale a evidențiat o eroare medie de 0.8°C pe întregul interval de măsurare, demonstrând precizia și fiabilitatea sistemului dual-ADC cu fuziune inteligentă. Această eroare redusă confirmă corectitudinea algoritmilor de conversie și a interpolării liniare între punctele de calibrare, validând abordarea adoptată pentru măsurarea temperaturii cu senzorul PT100.

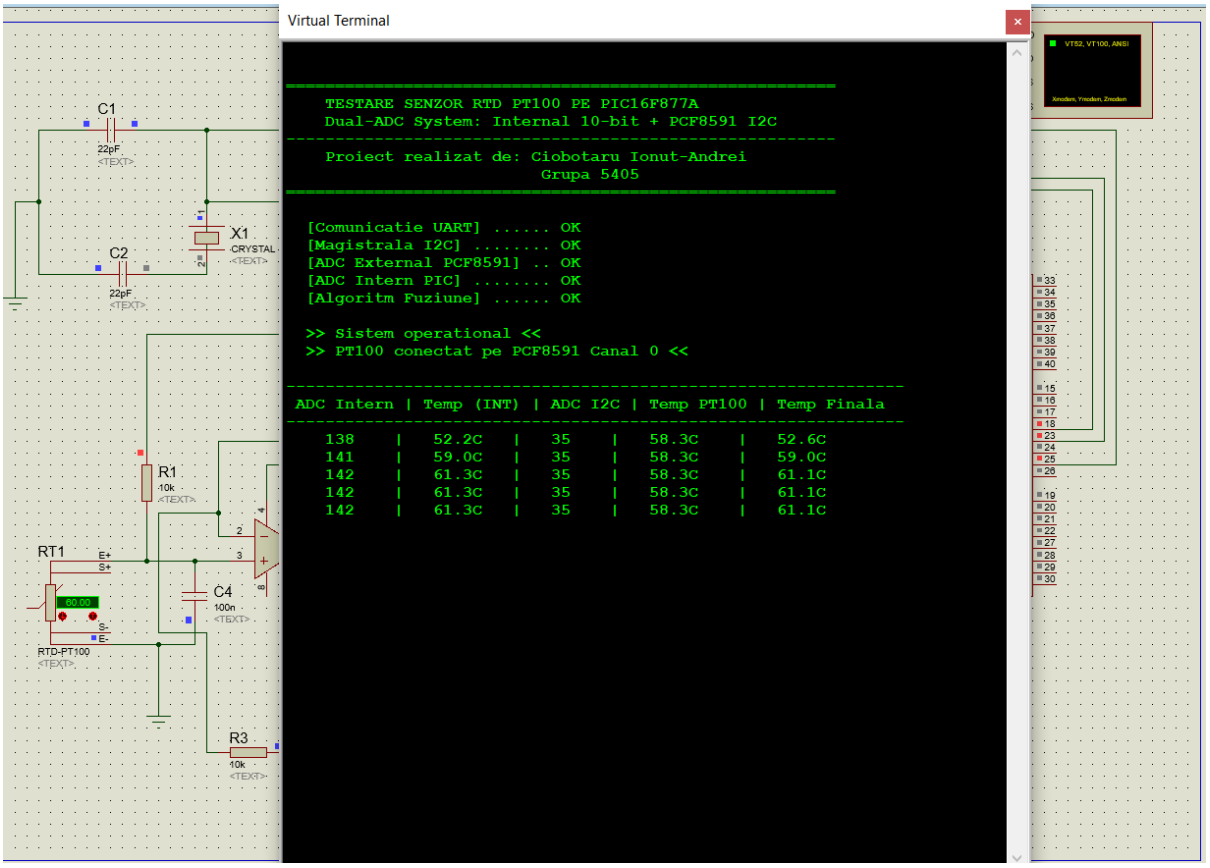


Figura 5.1 Rularea şi testarea sistemului pentru o valoare de 60 de grade celsius.

Din această imagine se observă funcționarea completă a sistemului de achiziție a temperaturii în simulare. Terminalul virtual afișează banner-ul de pornire cu titlul proiectului

și datele autorului, urmat de secvența de inițializare a tuturor modulelor: comunicația UART, magistrala I²C, ADC extern PCF8591, ADC intern PIC și algoritmul de fuziune, toate raportând status "OK".

Tabelul de date prezintă cinci citiri consecutive care evidențiază comportamentul sistemului la o temperatură stabilă în jurul valorii de 60°C. ADC-ul intern variază între 138 și 142 unități, convertite în temperaturi între 52.2°C și 61.3°C, în timp ce ADC-ul extern PCF8591 rămâne constant la valoarea 35, corespunzând la 58.3°C. Temperatura finală calculată de algoritmul de fuziune variază între 52.6°C și 61.1°C, adaptându-se la variațiile ADC-ului intern care are rezoluție superioară.

Temperatură RTD PT-100	Temperatură ADC Intern	Temperatură ADC Extern	Temperatură afișată	Eroare	Eroare medie
0,0	0,0	0,0	0,0	0,0	0,8
5,0	5,5	8,3	5,6	0,6	
10,0	11,1	8,3	10,9	0,9	
15,0	16,6	16,6	16,6	1,6	
20,0	22,2	16,6	22,0	2,0	
25,0	25,0	25,0	25,0	0,0	
30,0	31,8	25,0	30,1	0,1	
35,0	34,0	37,5	36,4	1,4	
40,0	40,9	37,5	40,7	0,7	
45,0	45,7	50,0	46,3	1,3	
50,0	50,0	50,0	50,0	0,0	
55,0	56,8	58,3	56,8	1,8	
60,0	61,3	58,3	61,1	1,1	
65,0	65,9	66,6	65,9	0,9	
70,0	70,5	66,6	70,4	0,4	
75,0	75,0	75,0	75,0	0,0	
80,0	81,8	87,5	82,1	2,1	
85,0	86,3	87,5	86,4	1,4	
90,0	88,6	87,5	88,5	-1,5	
95,0	95,4	100,0	96,3	1,3	
100,0	100,0	100,0	100,0	0,0	

Figura 5.2 Tabel realizat în excel cu valorile temperaturilor la un test cu pas de 5 grade celsius

Tabelul prezintă valorile experimentale obținute pe întregul interval de temperatură de la 0°C la 100°C, cu pas de 5°C între măsurători. Se observă că pe punctele de calibrare (0°C, 25°C, 50°C, 75°C, 100°C) eroarea este zero, ambele ADC-uri citind valorile exacte, iar algoritmul de fuziune returnează temperatura corectă.

În afara acestor puncte, apar diferențe între cele două surse: ADC-ul intern cu rezoluție de 10 biți oferă valori mai apropiate de temperatura reală, în timp ce ADC-ul extern de 8 biți prezintă salturi mai mari din cauza rezoluției reduse. Algoritmul de fuziune inteligentă combină cele două citiri, rezultând o temperatură finală cu o eroare medie de 0,8°C pe întregul interval, demonstrând eficiența sistemului dual-ADC. Cea mai mare eroare apare

la 80°C (2,1°C) unde există o diferență semnificativă între cele două surse, iar la 90°C apare o eroare negativă de -1,5°C, indicând o ușoară subestimare a temperaturii.

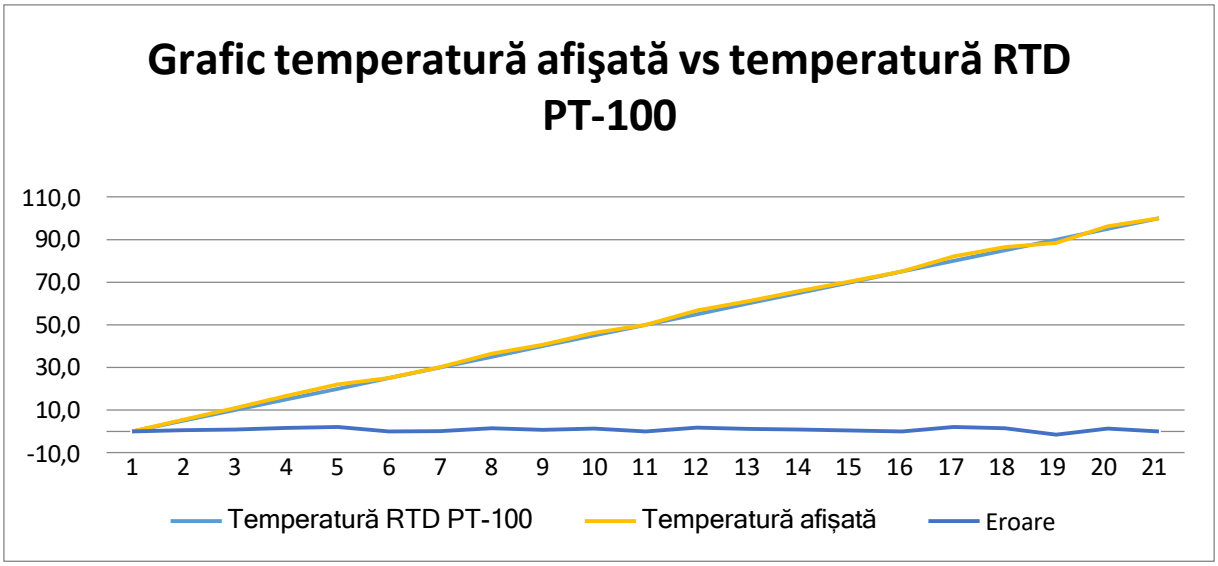


Figura 5.3. Grafic pentru diferenţa între temperatura afişată comparativ cu temperatura selectată de la RTD PT-100. În acest grafic este introdusă şi eroarea dintre temperatura afişată şi cea selectată

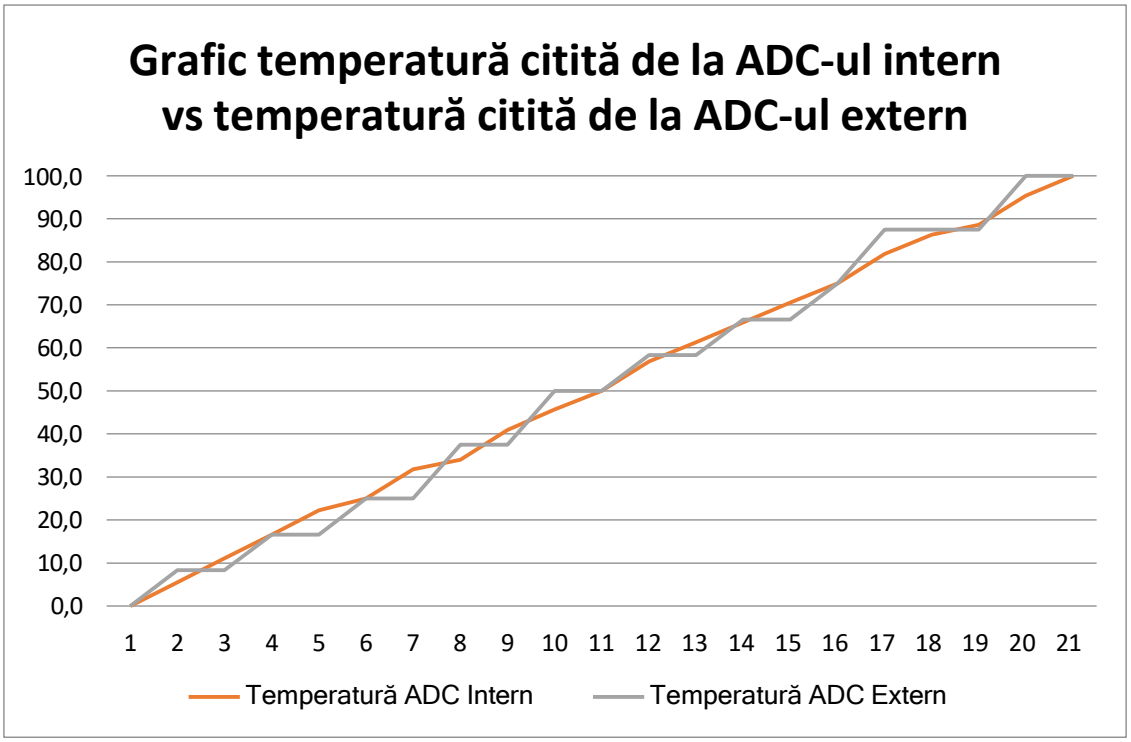


Figura 5.4. Grafic pentru diferenţa dintre temperatura citită cu ajutorul ADC-ului intern, al microprocesorului PIC16F877A şi temperatura citită cu ADC0-ul extern, PCF8591

Pe punctele de calibrare (1, 6, 11, 16 și 21 corespunzând la 0°C, 25°C, 50°C, 75°C și 100°C) cele două curbe se intersectează perfect, ambele citind valoarea corectă. Diferențele maxime între cele două surse apar în intervalele de mijloc între punctele de calibrare, unde

ADC-ul intern oferă valori mai apropiate de temperatura reală datorită capacității sale de a discerne variații mai mici.

Graficul demonstrează vizual necesitatea algoritmului de fuziune care compensează limitările ADC-ului extern prin valorificarea preciziei ADC-ului intern.

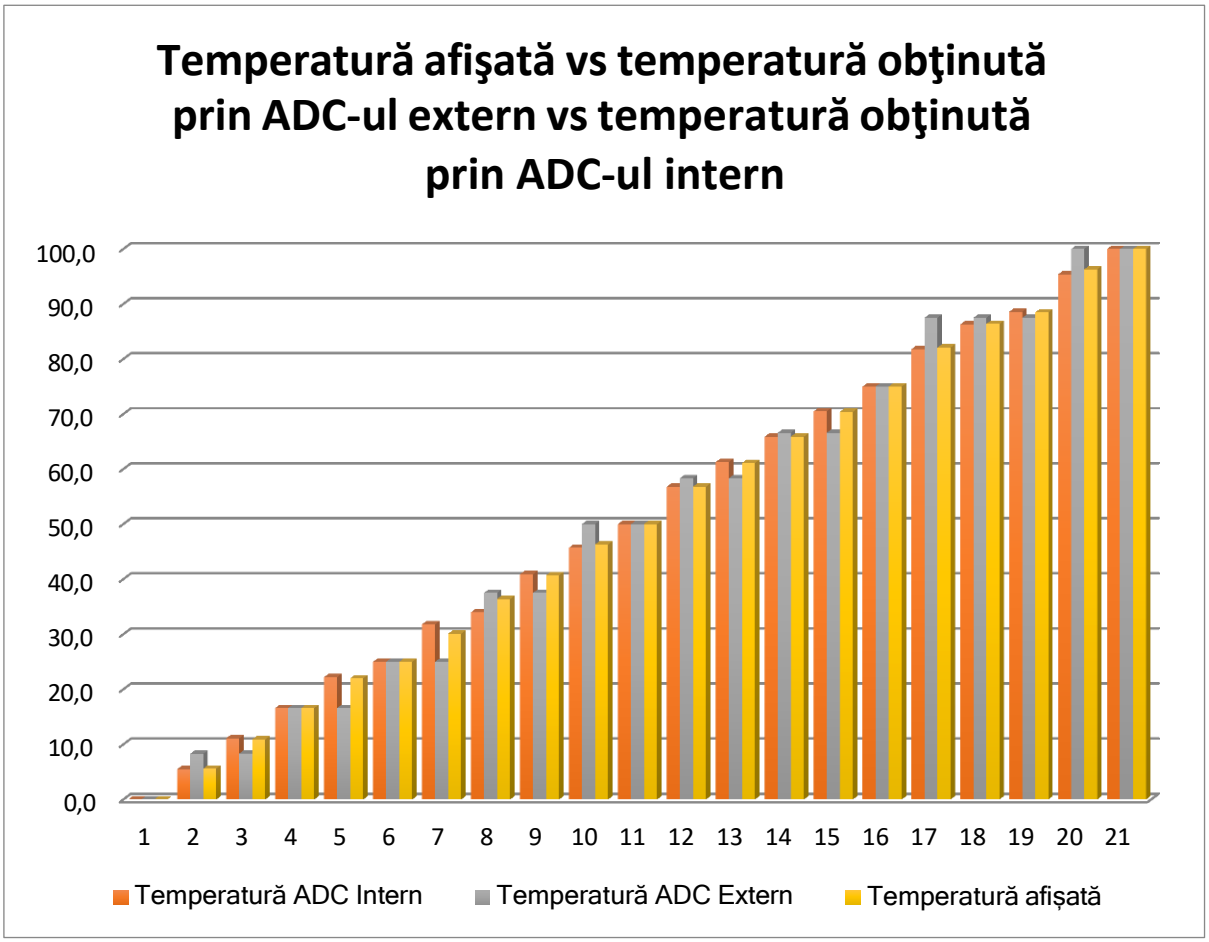


Figura 5.5. Grafic pentru diferenţa dintre temperatura citită cu ajutorul ADC-ului intern, al microprocesorului PIC16F877A şi temperatura citită cu ADC0-ul extern, PCF8591 şi temperatura

La adresa de mai jos se află un video încărcat pe YouTube cu funcționarea circuitului:

https://youtu.be/n3GuP8GxevM

Capitolul 6. Concluzii

În cadrul acestei lucrări a fost proiectat, implementat și validat un sistem de măsurare a temperaturii bazat pe senzorul RTD PT100, destinat aplicațiilor educaționale și experimentale, cu accent pe precizie, flexibilitate și accesibilitate. Sistemul propus îmbină o arhitectură hardware robustă, bazată pe punte Wheatstone, amplificare analogică și conversie analog–digitală dublă, cu o componentă software avansată care valorifică avantajele fiecărui convertor utilizat.

Utilizarea simultană a ADC-ului intern de 10 biți al microcontrolerului PIC16F877A și a convertorului extern PCF8591 de 8 biți, interfațat prin magistrala I²C, a permis implementarea unui algoritm de fuziune inteligentă capabil să combine informațiile provenite din surse cu rezoluții diferite. Această abordare a condus la obținerea unor rezultate mai stabile și mai apropiate de valoarea reală a temperaturii decât în cazul utilizării unui singur ADC.

Rezultatele obținute în urma testelor experimentale, desfășurate pe intervalul 0–100 °C, demonstrează corectitudinea metodei de calibrare, a interpolării liniare și a algoritmului de fuziune. Eroarea medie de aproximativ 0,8 °C confirmă faptul că sistemul oferă o precizie satisfăcătoare pentru aplicații de monitorizare și control al temperaturii, iar erorile maxime observate rămân în limite acceptabile, fiind explicabile prin rezoluția limitată a ADC-ului extern și prin interpolarea între punctele de calibrare.

Pe viitor, sistemul poate fi îmbunătățit prin utilizarea unui convertor analog–digital cu rezoluție mai mare, capabil să ofere o precizie superioară în achiziția semnalului provenit de la senzorul PT100. De asemenea, se poate implementa un algoritm de prelucrare mai avansat, bazat pe metode de compensare a neliniarităților sau pe tehnici statistice de filtrare și fuziune a datelor, cu scopul reducerii suplimentare a erorilor și creșterii acurateții măsurătorilor.

În concluzie, sistemul realizat validează posibilitatea obținerii unor măsurători precise utilizând componente uzuale și o arhitectură simplificată, demonstrând că soluțiile educaționale pot prelua concepte din sistemele industriale avansate. Proiectul poate constitui o bază solidă pentru dezvoltări ulterioare, precum extinderea domeniului de temperatură, integrarea unui ADC de rezoluție mai mare, implementarea compensării neliniarităților PT100 sau adăugarea unei interfețe grafice pentru vizualizarea datelor în timp real.

Capitolul 7. Bibliografie

[1] Process Parameters, “Pt100 Sensor – Working Principle.” [Online]. Available: https://www.processparameters.co.uk/pt100-sensor-working-principle/. [Accesat la data de: 23-Dec-2025].

[2] RealPars, “Pt100 Sensor Explained | Working Principles,” 2025. [Online]. Available: https://www.realpars.com/blog/pt100 [Accesat la data de: 23-Dec-2025]

[3] Rika Sensor, “The Advantages of PT100 Temperature Sensors in Industrial Applications,” 26-Aug-2023. [Online]. Available: https://www.rikasensor.com/the-advantages-of-pt100-temperature-sensors-in-industrial-applications.html. [Accesat la data de: 23-Dec-2025].

[4] El-Watch, “PT100 Sensors: What are they, and which types are best for your needs?”, 05-May-2023. [Online]. Available: https://el-watch.com/pt100-a-complete-guide/. [Accesat la data de: 23-Dec-2025].

[5] C. Bini, S. Ahmed, M. Del Bianco, “Temperature Probe and Method for Manufacturing a Temperature Probe,” European Patent EP 21215112.0, granted Feb. 5, 2025.

[6] M.T. Małek, H. Koczyk, Influence of Temperature Sensor (PT100) Accuracy on the Interpretation of Experimental Results of Measuring Temperature on the Surface, Civil and Environmental Engineering Reports, vol. 34, nr. 4, 2024, pp. 1–21, doi:10.59440/ceer/192146.

[7] Qing Wang et al., Simulation for Temperature Measurement and Control System Based on PT100, 2019

[8] Jiguang Liu, Yukun Li, Hongyan Zhao, A Temperature Measurement System Based on PT100, Liaoning Provincial College of Communications, China, 2010

[9] W. Walendziuk, A. Idzkowski, Z. Machacek, Z. Slanina, “Evaluation of Pt100 Sensor Deflection Effect during Strain Measurements,” Elektronika ir Elektrotechnika, vol. 21, no. 4, 2015.

[10] Q. Wang, G. Wang, X. Xie, L. Zhou, “Design and Simulation for Temperature Measurement and Control System Based on PT100,” Elektronika ir Elektrotehnika, vol. 21, no. 4, 2015.

[11] J. A. Prakosa, P. Purwowibowo, D. Larassati, “Development of Simple Method for Quality Testing of PT100 Sensors Due to Temperature Coefficient of Resistance Measurement,” IEEE ISESD, 2021.

[12]N. Olsson, *“Construction of a PT100 Sensor Simulator for CAN-Based Test Beds,”* BSc Thesis, Luleå University of Technology, Dept. Of Engineering Sciences and Mathematics, 2018.

[13] A. Chekichev, A. Parushev, A. Aleksandrov, D. Shehova, R. Popov, “High Precision Measurement of the Air Temperature Using Resistance Temperature Detector Pt100 Sensor,” Proc. 19th Int. Conf. ELMA, IEEE, 2025.

[14] X. Wang, X. Yin, Design of Signal Conditioning Module of PT100 Temperature Sensor Detection System, Key Engineering Materials, 428–429, 556–560, 2010.

[15] S. Jalnekar, V. Gaikwad, PIC16 Based Blowdown Controller for Industrial Boilers, ICDMAI 2017, IEEE.

[16] Heisener, "LM358N Operational Amplifier: Working Principle, Pinout and Equivalents," *Heisener.com*. [Online]. Available: https://www.heisener.com/TechnologyDetail/LM358N-Operational-Amplifier-Working-Principle-Pinout-and-Equivalents. [Accesat la data de: 23-Dec-2025].

[17] NXP Semiconductors, „PCF8591: 8-bit A/D and D/A converter, Rev. 7, 27 June 2013," Product Data Sheet. [Online]. Available: https://www.nxp.com/docs/en/data-sheet/PCF8591.pdf. [Accesat la data de: 23-Dec-2025].

[18] Optimus Digital S.R.L., "Modul Convertor AD/DA PCF8591 cu Fotorezistor și Termistor," *OptimusDigital.ro*. [Online]. Available: https://www.optimusdigital.ro/ro/convertoare-analogic-numerice-adc/170-modul-convertor-adda-pcf8591.html. [Accesat la data de: 23-Dec-2025].

[19] Microchip Technology Inc., „PIC16F87XA 28/40/44-Pin Enhanced Flash Microcontrollers," Microchip Technology, Datasheet, 2006. [Online]. Available: https://www.microchip.com/wwwproducts/en/PIC16F87XA

[20] S. Smith, "PIC16F877A Pinout and Pin Configuration Guide," *SUNPCB*, Apr. 19, 2025. [Online]. Available: https://www.sunpcb.com/blog/pic16f877a-pinout-and-pin-configuration-guide.html

[21] J. Wu, A Basic Guide to I2C, Texas Instruments, Application Note SBAA565, 2011. [Online]. Available: https://www.ti.com/lit/an/sbaa565/sbaa565.pdf?ts=1766454295692&ref_url=https%253A%252F%252Fwww.google.com%252F

[22] S. Campbell, Basics of the I2C Communication Protocol, Circuit Basics, Apr. 28, 2023. [Online]. Available: https://www.circuitbasics.com/basics-of-the-i2c-communication-protocol/ [Accesat la data de: 23-Dec-2025]

[23] E. Peña și M. G. Legaspi, "UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter," *Vol. 54, No. 4*, Dec. 2020.

Anexă. Codul sursă al proiectului:

```
#include <xc.h>
#include <stdio.h>
#include <string.h>

#define _XTAL_FREQ 8000000

#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = OFF
#pragma config BOREN = OFF
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config WRT = OFF
#pragma config CP = OFF

// I2C addresses
#define  PCF8591_ADDR   0x90
#define  PCF8591_WRITE   0x90
#define PCF8591_READ   0x91

// Calibrare ADC INTERN (10-bit: 0-1023)
#define ADC_INT_AT_0C   117
#define ADC_INT_AT_25C   126
#define ADC_INT_AT_50C   137
#define ADC_INT_AT_75C   148
#define ADC_INT_AT_100C  159

// Calibrare PT100 prin PCF8591 (8-bit: 0-255)
#define ADC_PT100_AT_0C     29
#define ADC_PT100_AT_25C    32
#define ADC_PT100_AT_50C    34
#define ADC_PT100_AT_75C    37
#define ADC_PT100_AT_100C   39

// ==========================================================================
```

```
// UART FUNCTIONS
// ==============================================================================
void UART_Init(void)
{
   TRISC6 = 0;
   TRISC7 = 1;
   SPBRG = 25;
   TXSTA = 0x24;
   RCSTA = 0x90;
}

void UART_Write(char data)
{
   while(!TXIF);
   TXREG = data;
}

void UART_Write_String(const char *str)
{
   while(*str)
      UART_Write(*str++);
}

void UART_Write_Int(int value)
{
   char buffer[12];
   sprintf(buffer, "%d", value);
   UART_Write_String(buffer);
}

// ==============================================================================
// I2C FUNCTIONS
// ==============================================================================
void I2C_Init(void)
{
   TRISC3 = 1;
```

```
    TRISC4 = 1;
    SSPSTAT = 0x80;
    SSPCON = 0x28;
    SSPCON2 = 0x00;
    SSPADD = 19;
    __delay_ms(10);
}

void I2C_Start(void)
{
    SEN = 1;
    while(SEN);
}

void I2C_Stop(void)
{
    PEN = 1;
    while(PEN);
}

void I2C_Wait(void)
{
    while((SSPCON2 & 0x1F) || (SSPSTAT & 0x04));
}

void I2C_Write(unsigned char data)
{
    SSPBUF = data;
    while(BF);
    I2C_Wait();
}

unsigned char I2C_Read(unsigned char ack)
{
    unsigned char data;
```

```
    RCEN = 1;
    while(!BF);
    data = SSPBUF;

    I2C_Wait();

    ACKDT = ack ? 0 : 1;
    ACKEN = 1;
    while(ACKEN);

    return data;
}

// ==============================================================================
// PCF8591 ADC FUNCTIONS
// ==============================================================================
unsigned char PCF8591_Read_Channel(unsigned char channel)
{
    unsigned char control_byte, data1, data2;

    control_byte = 0x40 | (channel & 0x03);

    I2C_Start();
    I2C_Write(PCF8591_WRITE);
    I2C_Write(control_byte);
    I2C_Stop();

    __delay_us(100);

    I2C_Start();
    I2C_Write(PCF8591_READ);
    data1 = I2C_Read(1);
    data2 = I2C_Read(0);
    I2C_Stop();

    return data2;
```

```
}

unsigned int PCF8591_Read_Avg(unsigned char channel)
{
    unsigned long sum = 0;
    unsigned char i;

    for(i = 0; i < 32; i++) {
        sum += PCF8591_Read_Channel(channel);
        __delay_ms(3);
    }

    return (unsigned int)(sum >> 5);
}

// ==============================================================================
// INTERNAL ADC FUNCTIONS
// ==============================================================================
void ADC_Internal_Init(void)
{
    TRISA0 = 1;
    ADCON0 = 0x81;
    ADCON1 = 0x8E;
    __delay_ms(1);
}

unsigned int ADC_Internal_Read(void)
{
    __delay_us(20);
    ADCON0bits.GO = 1;
    while(ADCON0bits.GO);
    return ((ADRESH << 8) | ADRESL);
}

unsigned int ADC_Internal_Read_Avg(void)
{
```

```
    unsigned long sum = 0;
    unsigned char i;

    for(i = 0; i < 32; i++) {
        sum += ADC_Internal_Read();
        __delay_ms(3);
    }

    return (unsigned int)(sum >> 5);
}

// ==============================================================================
// TEMPERATURE CONVERSION - ADC INTERN
// ==============================================================================
int Get_Temp_Internal_Int(unsigned int adc)
{
    long temp_x100;

    if(adc <= ADC_INT_AT_25C) {
        temp_x100 = ((long)(adc - ADC_INT_AT_0C) * 2500L) /
                (ADC_INT_AT_25C - ADC_INT_AT_0C);
    }
    else if(adc <= ADC_INT_AT_50C) {
        temp_x100 = 2500 + ((long)(adc - ADC_INT_AT_25C) * 2500L) /
                (ADC_INT_AT_50C - ADC_INT_AT_25C);
    }
    else if(adc <= ADC_INT_AT_75C) {
        temp_x100 = 5000 + ((long)(adc - ADC_INT_AT_50C) * 2500L) /
                (ADC_INT_AT_75C - ADC_INT_AT_50C);
    }
    else {
        temp_x100 = 7500 + ((long)(adc - ADC_INT_AT_75C) * 2500L) /
                (ADC_INT_AT_100C - ADC_INT_AT_75C);
    }

    if(temp_x100 < 0) temp_x100 = 0;
```

```
    if(temp_x100 > 10000) temp_x100 = 10000;

    return (int)temp_x100;
}

// ==============================================================================
// TEMPERATURE CONVERSION - PT100
// ==============================================================================
int Get_Temp_PT100_Int(unsigned int adc)
{
    long temp_x100;
    float adc_f = (float)adc;

    if(adc <= ADC_PT100_AT_25C) {
        float ratio = (adc_f - (float)ADC_PT100_AT_0C) /
                    ((float)ADC_PT100_AT_25C - (float)ADC_PT100_AT_0C);
        temp_x100 = (long)(ratio * 2500.0);
    }
    else if(adc <= ADC_PT100_AT_50C) {
        float ratio = (adc_f - (float)ADC_PT100_AT_25C) /
                    ((float)ADC_PT100_AT_50C - (float)ADC_PT100_AT_25C);
        temp_x100 = 2500 + (long)(ratio * 2500.0);
    }
    else if(adc <= ADC_PT100_AT_75C) {
        float ratio = (adc_f - (float)ADC_PT100_AT_50C) /
                    ((float)ADC_PT100_AT_75C - (float)ADC_PT100_AT_50C);
        temp_x100 = 5000 + (long)(ratio * 2500.0);
    }
    else {
        float ratio = (adc_f - (float)ADC_PT100_AT_75C) /
                    ((float)ADC_PT100_AT_100C - (float)ADC_PT100_AT_75C);
        temp_x100 = 7500 + (long)(ratio * 2500.0);
    }

    if(temp_x100 < 0) temp_x100 = 0;
    if(temp_x100 > 10000) temp_x100 = 10000;
```

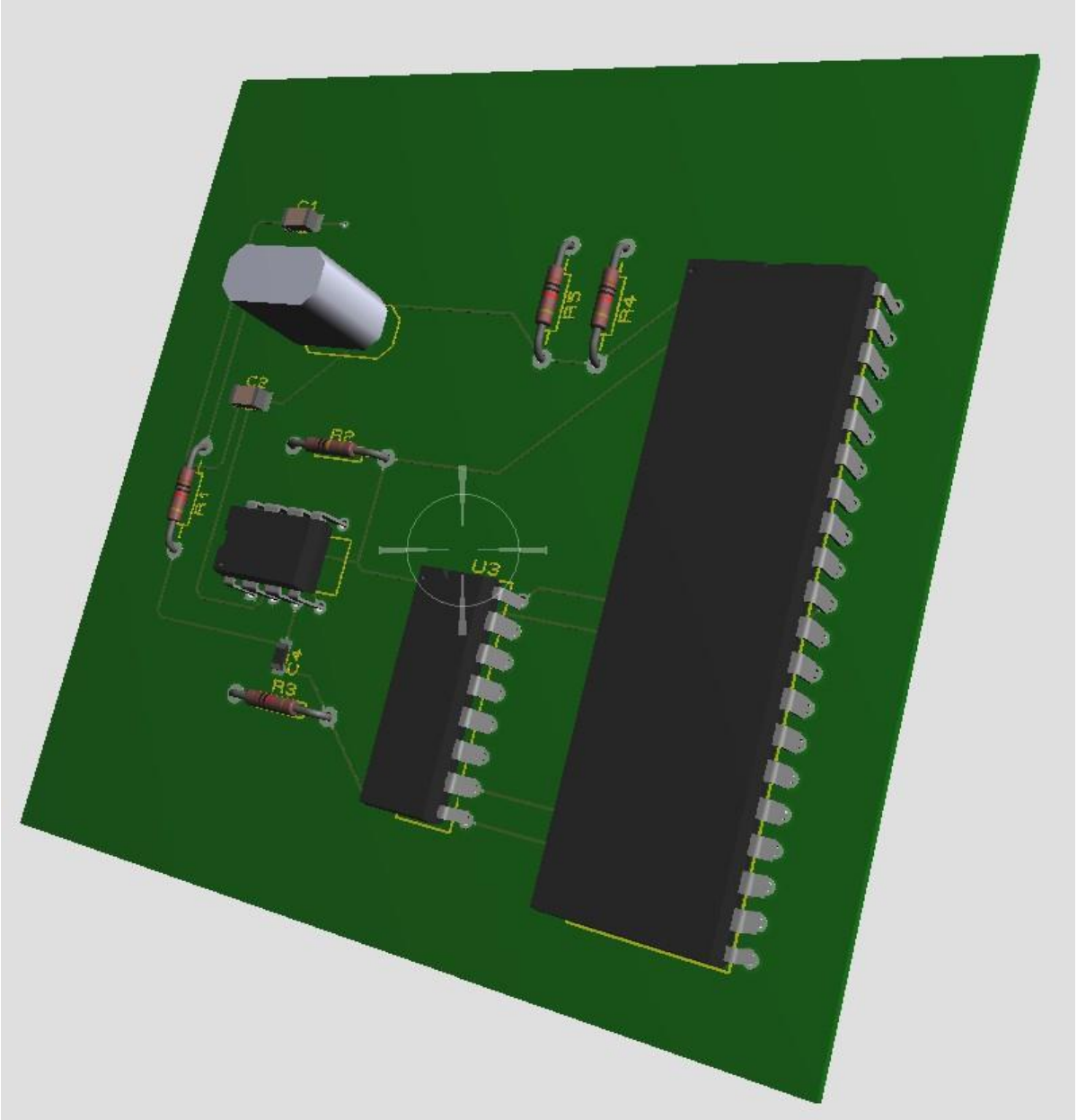
```
    return (int)temp_x100;
}

// ==============================================================================
// FUZIUNE INTELIGENTĂ - Selectează cea mai bună temperatură
// ==============================================================================
int Get_Smart_Temperature(unsigned int adc_int, int temp_int,
                          unsigned int adc_pt100, int temp_pt100,
                          unsigned char *source_id)
{
    float confidence_int = 1.0;
    float confidence_pt100 = 1.0;

    // ADC Intern: verifică proximitatea fa?ă de puncte de calibrare
    if((adc_int >= ADC_INT_AT_0C - 2 && adc_int <= ADC_INT_AT_0C + 2) ||
       (adc_int >= ADC_INT_AT_25C - 2 && adc_int <= ADC_INT_AT_25C + 2) ||
       (adc_int >= ADC_INT_AT_50C - 2 && adc_int <= ADC_INT_AT_50C + 2) ||
       (adc_int >= ADC_INT_AT_75C - 2 && adc_int <= ADC_INT_AT_75C + 2) ||
       (adc_int >= ADC_INT_AT_100C - 2 && adc_int <= ADC_INT_AT_100C + 2)) {
       confidence_int = 1.0;
    } else {
        confidence_int = 0.8;
    }

    // PT100: verifică dacă e EXACT pe punct de calibrare
    if(adc_pt100 == ADC_PT100_AT_0C ||
       adc_pt100 == ADC_PT100_AT_25C ||
       adc_pt100 == ADC_PT100_AT_50C ||
       adc_pt100 == ADC_PT100_AT_75C ||
       adc_pt100 == ADC_PT100_AT_100C) {
       confidence_pt100 = 1.0;
    } else {
        confidence_pt100 = 0.2;
    }
```

```
    // Factor de calitate: ADC intern are 4x mai multă rezolu?ie
    confidence_int *= 4.0;

    // Determină sursa
    float total_confidence = confidence_int + confidence_pt100;

    if(confidence_pt100 >= 0.9 && confidence_int >= 3.5) {
        *source_id = 3;  // BOTH
    } else if(confidence_int > confidence_pt100 * 2.0) {
        *source_id = 1;  // INT
    } else {
        *source_id = 2;  // MIXED
    }

    // Calculează temperatura finală (medie ponderată)
    long temp_smart = ((long)temp_int * (long)(confidence_int * 100.0) +
                       (long)temp_pt100 * (long)(confidence_pt100 * 100.0)) /
                       (long)(total_confidence * 100.0);

    return (int)temp_smart;
}

// ==============================================================================
// DISPLAY FUNCTIONS
// ==============================================================================
void Print_Startup_Banner(void)
{
    UART_Write_String("\r\n\r\n");
    UART_Write_String("=========================================================\r\n");
    UART_Write_String("   TESTARE SENZOR RTD PT100 PE PIC16F877A\r\n");
    UART_Write_String("   Dual-ADC System: Internal 10-bit + PCF8591 I2C\r\n");
    UART_Write_String("-------------------------------------\r\n");
    UART_Write_String("   Proiect realizat de: Ciobotaru Ionut-Andrei\r\n");
    UART_Write_String("                        Grupa 5405\r\n");
    UART_Write_String("=========================================================\r\n\r\n");
}
```

```
void Print_Init_Status(const char* module, const char* status)
{
    UART_Write_String("  [");
    UART_Write_String(module);
    UART_Write_String("] ");

    unsigned char i;
    for(i = 0; i < (22 - strlen(module)); i++) {
        UART_Write('.');
    }

    UART_Write(' ');
    UART_Write_String(status);
    UART_Write_String("\r\n");
}

void Print_Table_Header(void)
{
    UART_Write_String("\r\n");
    UART_Write_String("----------------------------------------\r\n");
    UART_Write_String(" ADC Intern | Temp (INT) | ADC I2C | Temp PT100 | Temp Finala \r\n");
    UART_Write_String("----------------------------------------\r\n");
}

void Print_Reading(unsigned int adc_int, int temp_int,
                   unsigned int adc_pt100, int temp_pt100,
                   int temp_best, unsigned char source)
{
    // ADC Intern (10-bit)
    UART_Write_String("   ");
    if(adc_int < 100) UART_Write(' ');
    if(adc_int < 10) UART_Write(' ');
    UART_Write_Int(adc_int);
    UART_Write_String("   |   ");
```

```
// Temp Intern
if(temp_int < 1000) UART_Write(' ');
UART_Write_Int(temp_int / 100);
UART_Write('.');
UART_Write(((temp_int / 10) % 10) + '0');
UART_Write_String("C  |  ");

// ADC I2C (8-bit)
if(adc_pt100 < 100) UART_Write(' ');
if(adc_pt100 < 10) UART_Write(' ');
UART_Write_Int(adc_pt100);
UART_Write_String("  |  ");

// Temp I2C
if(temp_pt100 < 1000) UART_Write(' ');
UART_Write_Int(temp_pt100 / 100);
UART_Write('.');
UART_Write(((temp_pt100 / 10) % 10) + '0');
UART_Write_String("C  |  ");

// Temperatura Finală (smart fusion)
if(temp_best < 1000) UART_Write(' ');
UART_Write_Int(temp_best / 100);
UART_Write('.');
UART_Write(((temp_best / 10) % 10) + '0');
UART_Write_String("C");

// Indicator calitate
if(source == 3) {
   UART_Write_String(" *");  // Ambele pe calibrare - maxim de încredere
} else if(source == 1) {
   UART_Write_String("  ");  // ADC intern dominant
} else {
   UART_Write_String(" ~");  // Medie ponderată
}
```

```
    UART_Write_String("\r\n");
}

// ==============================================================================
// MAIN PROGRAM
// ==============================================================================
void main(void)
{
    unsigned int adc_int, adc_pt100;
    int temp_int_x100, temp_pt100_x100, temp_best_x100;
    unsigned char source_id;

    // Initialize
    UART_Init();
    __delay_ms(100);

    Print_Startup_Banner();

    Print_Init_Status("Comunicatie UART", "OK");
    __delay_ms(200);

    Print_Init_Status("Magistrala I2C", "OK");
    I2C_Init();
    __delay_ms(200);

    Print_Init_Status("ADC External PCF8591", "OK");
    __delay_ms(200);

    Print_Init_Status("ADC Intern PIC", "OK");
    ADC_Internal_Init();
    __delay_ms(200);

    Print_Init_Status("Algoritm Fuziune", "OK");
    __delay_ms(200);

    UART_Write_String("\r\n");
```

```
    UART_Write_String("  >> Sistem operational <<\r\n");
    UART_Write_String("  >> PT100 conectat pe PCF8591 Canal 0 <<\r\n");

    Print_Table_Header();

    while(1)
    {
        // Cite?te de la ambele surse
        adc_int = ADC_Internal_Read_Avg();
        adc_pt100 = PCF8591_Read_Avg(0);

        // Calculează temperaturi
        temp_int_x100 = Get_Temp_Internal_Int(adc_int);
        temp_pt100_x100 = Get_Temp_PT100_Int(adc_pt100);

        // Fuziune inteligentă
        temp_best_x100 = Get_Smart_Temperature(adc_int, temp_int_x100,
                                    adc_pt100, temp_pt100_x100,
                                    &source_id);

        Print_Reading(adc_int, temp_int_x100,
                    adc_pt100, temp_pt100_x100,
                    temp_best_x100, source_id);

        __delay_ms(1000);
    }
}
```

Anexă. Imaginea 3D a sistemului